File: `src/hooks/useShellyStatus.js`

Category: Custom React Hook (Polling + Data History)
Runtime: Browser (React)
Primary consumer: `src/components/ShellyDashboard.jsx`
Upstream dependencies:

- `src/api/shellyClient.js` (`fetchShellyStatus`)
- `src/domain/parseShellyData.js` (`parseShellyData`)

Overview

`useShellyStatus` is the application's **data acquisition hook**. It:

1. Polls the Shelly RPC endpoint on a fixed interval
2. Parses the raw JSON into a normalized `ParsedDataPoint`
3. Stores a rolling history array (bounded by `maxPoints`)
4. Returns that history for charts and readouts

This hook is intentionally responsible for **polling and storage only**. It does not filter by interval, compute statistics, or format values—those are handled by components.

Imports

- `useState`, `useEffect`, `useRef` from React
- `fetchShellyStatus` from `../api/shellyClient`
- `parseShellyData` from `../domain/parseShellyData`

Exported API

`useShellyStatus(intervalMs = 2000, maxPoints = 2000)`

Signature

```
useShellyStatus(intervalMs?: number, maxPoints?: number) → ParsedDataPoint[]
```

Parameters

- `intervalMs` (number, default `2000`)
  Polling period in milliseconds. Example: `2000` means “poll every 2 seconds”.
- `maxPoints` (number, default `2000`)
  Maximum number of historical samples retained. Oldest samples are discarded once the limit is exceeded.

Returns

- `dataHistory`: `ParsedDataPoint[]`
  Chronological array of parsed samples (oldest → newest).

Internal State and References

State: `dataHistory`

```
const [dataHistory, setDataHistory] = useState([]);
```

- Holds the rolling history of samples.
- Updating this state triggers re-renders of any component using the hook.

Reference: `timerRef`

```
const timerRef = useRef(null);
```

- Stores the interval timer ID returned by `setInterval`.
- A reference is used so the timer ID persists across renders without causing re-renders.

Polling Lifecycle (`useEffect`)

Effect structure

```
useEffect(callback, [intervalMs, maxPoints])
```

Dependency array: `[intervalMs, maxPoints]`

The effect is re-executed when either value changes:

- If `intervalMs` changes: the polling timer must restart with a new period.
- If `maxPoints` changes: the retention logic must apply the new limit.

This ensures the hook remains correct even if the caller dynamically changes polling frequency or history size (***currently not used in this testing version***).

Effect steps (runtime behavior)

1. Define an internal asynchronous function `update()` that:
   - Calls `fetchShellyStatus()` to retrieve raw JSON
   - Calls `parseShellyData(raw)` to normalize the payload
   - Appends the result to `dataHistory`
   - Enforces `maxPoints` by removing oldest entries
2. Call `update()` once immediately
   - This avoids waiting one full interval before the first datapoint arrives.
3. Start recurring polling:

   ```
   timerRef.current = setInterval(update, intervalMs);
   ```

4. Cleanup function:

   ```
   return () => clearInterval(timerRef.current);
   ```

   Ensures the timer is removed when:

   - the component unmounts, or
   - `intervalMs/maxPoints` changes (effect re-run)

State update strategy (important detail)

Inside `update()`:

```
setDataHistory(prev => {
  const next = [...prev, parsed];
  if (next.length > maxPoints) next.shift();
  return next;
});
```

This is the “functional setState” form.

Why it is used:

- Prevents stale state bugs in asynchronous logic
- Ensures updates are based on the latest `prev` value even when multiple updates occur over time

Retention behavior:

- When the history exceeds `maxPoints`, the oldest element is removed using `shift()`.

Error handling

If fetching fails:

```
catch (err) {
  console.error("Shelly fetch error:", err);
}
```

Effects of errors:

- No new datapoint is added for that poll cycle
- The UI remains running with previous data
- Failures are visible in the browser console for debugging

Side effects

- Creates a repeating timer (`setInterval`)
- Performs network requests repeatedly
- Writes errors to the console

Common edge cases

- If `parseShellyData(raw)` returns `null`, a `null` entry would be appended.
  (In your current implementation, this is unlikely because raw is expected to exist, but it is a possible scenario if the API returns unexpected data. If you want, we can add a guard: `if (!parsed) return;`.)
- If `intervalMs` is very small and `maxPoints` very large, memory usage increases. The bounded history is what prevents unbounded growth.